Vaidehi Dalmia - HW2

TASK - MuJoCo PandaPickCube Environment
The Franka has to pick up the red cube and move it to the target location marked in green. It has to keep the cube at the target location till the end of the episode.

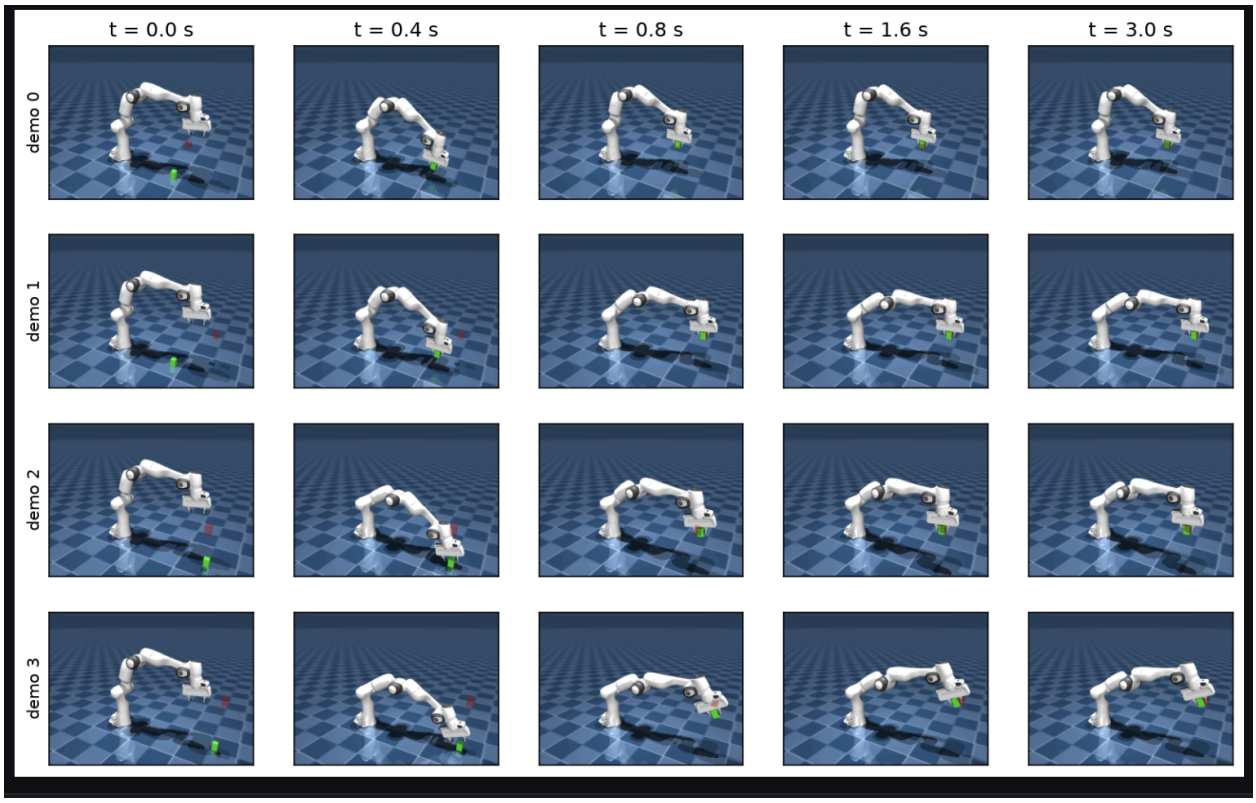
EVALUATION METRICS -
At the end of the episode (150 steps), is the cube within 0.02m (euclidean distance, rotation not taken into account) of the target location.

TEACHER -
PPO using the same code as MuJoCo playground Manipulation. I changed the environment to PickCube and impl:jax and kept the default config.
Demonstrations were generated by running the PPO teacher on 1050 episodes. The dataset contains both successful and unsuccessful demonstrations but only the first n successful demonstrations were selected to train the student policies below.

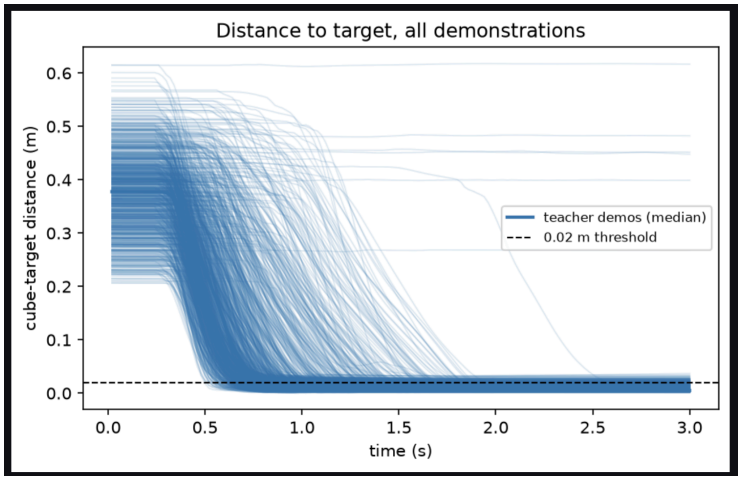
DATASET QUALITY -
The dataset stats are saved in results/dataset_stats_pool.json and videos in videos/demos_pool. The teacher has a success rate of 90.8% when collecting the data with final_dist median = 0.005m, well below the 0.02m success threshold. The trajectory length is always 150 because the episode has a fixed horizon of 150 steps, and episodes only end early if the cube goes out of bounds, which never happened in the demos. Steps-to-success is 0.94 ± 0.32 s.

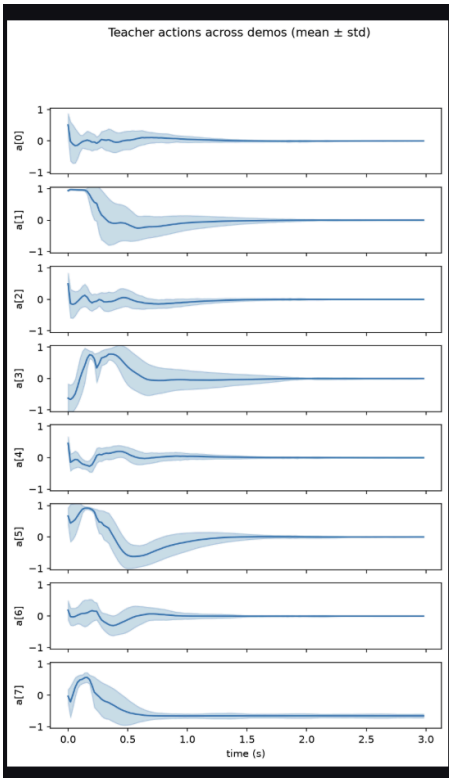

```
{
 "config": {
   "demos_seed": 0,
   "n_episodes": 1050,
   "n_videos": 5,
   "out": "data/demos_pool_1000.npz"
 },
 "teacher":
"/home/vaidehi/csci645/DALMIA_VAIDEHI_HW2/checkpoints/teacher/000024576000",
 "n_episodes": 1050,
 "success_rate": 0.9076190476190477,
 "ever_reached_rate": 0.9114285714285715,
 "reached_box_rate": 0.9942857142857143,
 "final_dist_median": 0.0051996055990457535,
 "final_dist_max": 0.6167649030685425,
 "final_success_start_s_mean": 0.9430849947534102,
 "final_success_start_s_std": 0.3191241458709848,
 "episode_lengths": [
   150
```

```
 ],
 "n_successful": 953
}
```



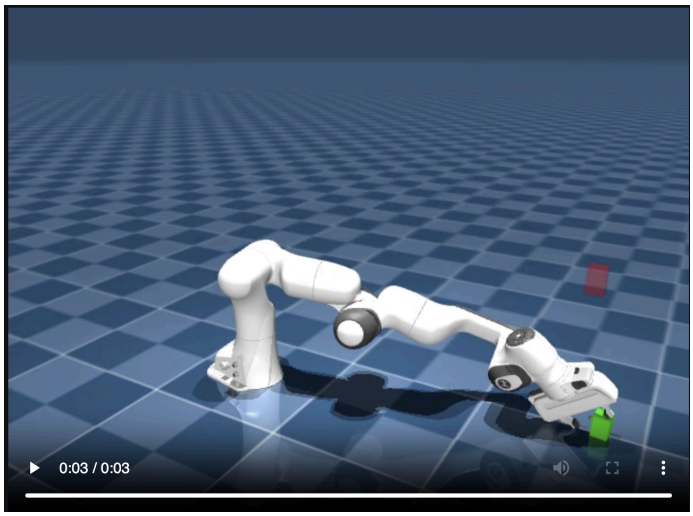
Demo 3 in the above picture is a failure (cube was above the threshold distance), the rest are successes.





The successful demos follow a pretty consistent pattern with a fast approach and grasp in the first ~0.3 s, transport to the target, then holding still with near-zero actions. The action spread across demos is widest during the approach and grasp, and close to zero during the hold, so demos differ mainly in how they get to the target, not in how they hold it.

Failures include being unable to pick up the cube (diagram shows a flat line), or cube being held away from the 0.02m threshold.



Failure because the teacher was unable to pick up the cube - it looks like the cube is almost out of reach.

STUDENT POLICY AND TRAINING -
I trained a Diffusion Policy following the Diffusion Policy Colab. I used the first 100 successful episodes from the dataset.

Inputs - The observation space is 66 dimensions. All joint positions, cube position, gripper position, gripper orientation, cube orientation, vector from gripper to cube, vector from cube to target, orientation error, how far each actuator's position target is from the current joint position

```
def _get_obs(self, data: mjx.Data, info: dict[str, Any]) -> jax.Array:
  gripper_pos = data.site_xpos[self._gripper_site]
  gripper_mat = data.site_xmat[self._gripper_site].ravel()
  target_mat = math.quat_to_mat(data.mocap_quat[self._mocap_target])
  obs = jp.concatenate([
      data.qpos,
      data.qvel,
      gripper_pos,
      gripper_mat[3:],
      data.xmat[self._obj_body].ravel()[3:],
      data.xpos[self._obj_body] - data.site_xpos[self._gripper_site],
      info["target_pos"] - data.xpos[self._obj_body],
      target_mat.ravel()[:6] - data.xmat[self._obj_body].ravel()[:6],
      data.ctrl - data.qpos[self._robot_qposadr[:-1]],
  ])

  return obs
```

Outputs - 8 dimensional delta change joint actions * 16 action chunks
Observations and actions are min-max normalized to [-1, 1] using each training subset's stats. Actions are mapped back to action units clipped to [-1, 1].
Action representation and execution - The policy executes 4 actions from each chunk, then plans again from the new observations. At each step, the environment multiplies the action by 0.04 and adds it to the previous joint position targets, so actions are incremental changes to the joint targets, applied at 50 Hz.
Training Objective - DDPM noise-prediction objective - For each training sample, Gaussian noise at a randomly chosen diffusion step (100 steps, squared-cosine schedule) is added to a normalized 16-step action chunk. A conditional 1D U-Net, conditioned on the two most recent observations through FiLM, is trained to predict the added noise using mean squared error. At inference, an action chunk is generated by starting from pure noise and applying the network for 100 denoising steps.
Training Details - AdamW, lr 1e-4, weight decay 1e-6, batch 256, cosine schedule with 500 warmup steps, 30,000 gradient steps, EMA weights for eval, training seed 0, U-Net dims [256, 512, 1024]

Training Config -

configs > ! student_demos100.yaml

```yaml
condition: demos100
seed: 0
checkpoint_dir: checkpoints/student_demos100

data:
  path: data/demos_pool_1000.npz
  episodes: first_n_successful
  n_demos: 100
  pred_horizon: 16
  obs_horizon: 2
  action_horizon: 4

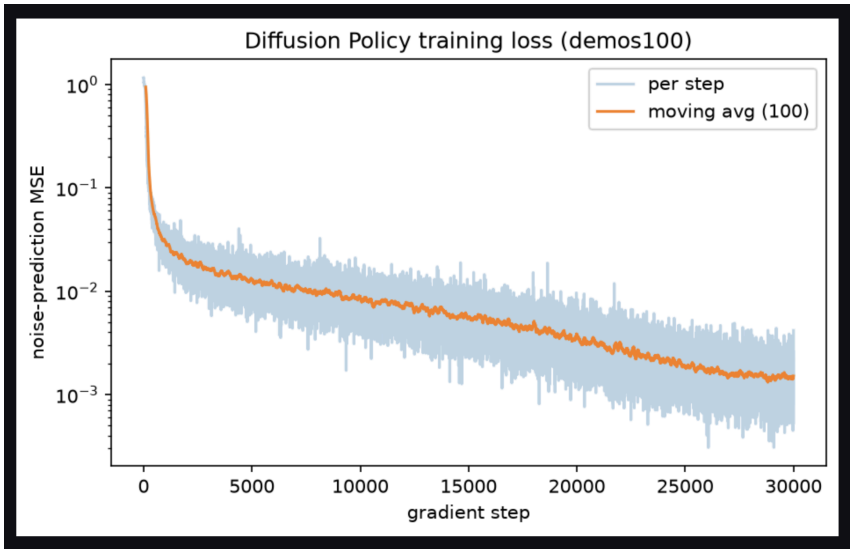
model:
  down_dims: [256, 512, 1024]
  diffusion_step_embed_dim: 256
  kernel_size: 5
  n_groups: 8

diffusion:
  num_train_timesteps: 100
  beta_schedule: squaredcos_cap_v2
  clip_sample: true
  prediction_type: epsilon

training:
  batch_size: 256
  num_steps: 30000
  lr: 1.0e-4
  weight_decay: 1.0e-6
  warmup_steps: 500
  ema_power: 0.75
  num_workers: 0

wandb:
  enabled: false
  mode: offline
  project: csci645-hw2
```

Loss Curve -



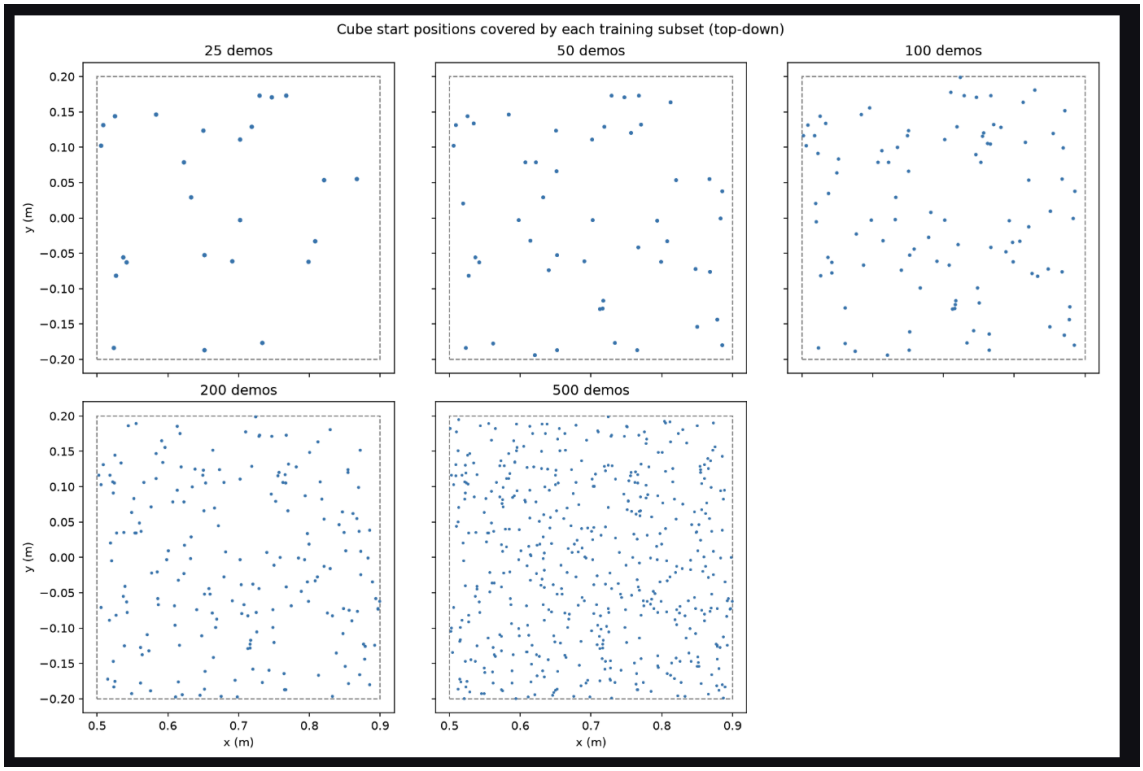
BASELINE RESULTS -

18.7% over 3 seeds and 50 rollouts for each seed = 150 rollouts total. Rollout videos saved under videos/student_demos100_es1000. Results analyzed with ablation below.

DATA ABLATION -

From initial experiments, I realized that the coverage was pretty low with 100 demonstrations. I wanted to see if more demonstrations would improve the success rate.

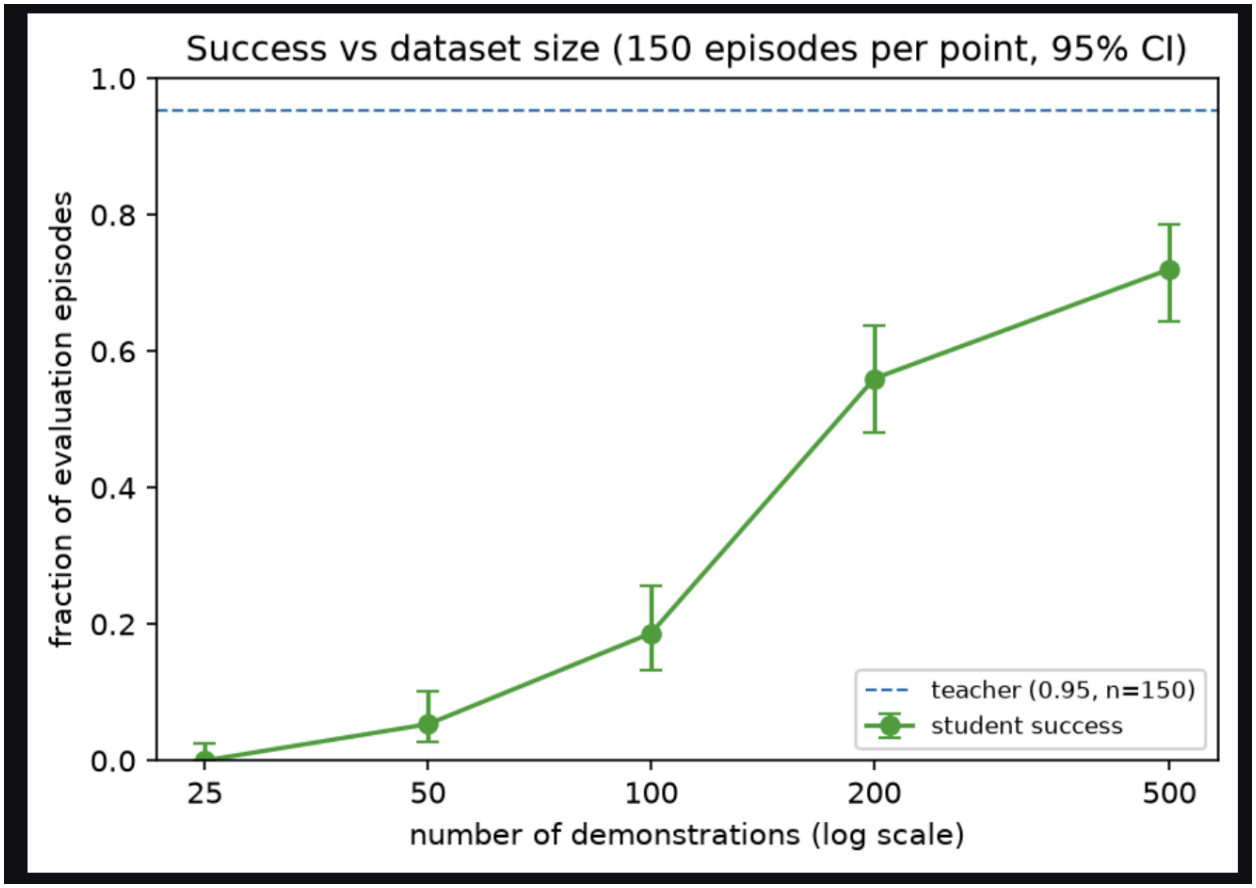
I tested with 25, 50, 100, 200, 500 first successful demonstrations from the pool of 1,050 episodes (953 successful). Everything else in the training config was kept the same. 30,000 gradient steps for every size (about 2,143 epochs at 25 demos down to about 111 at 500).

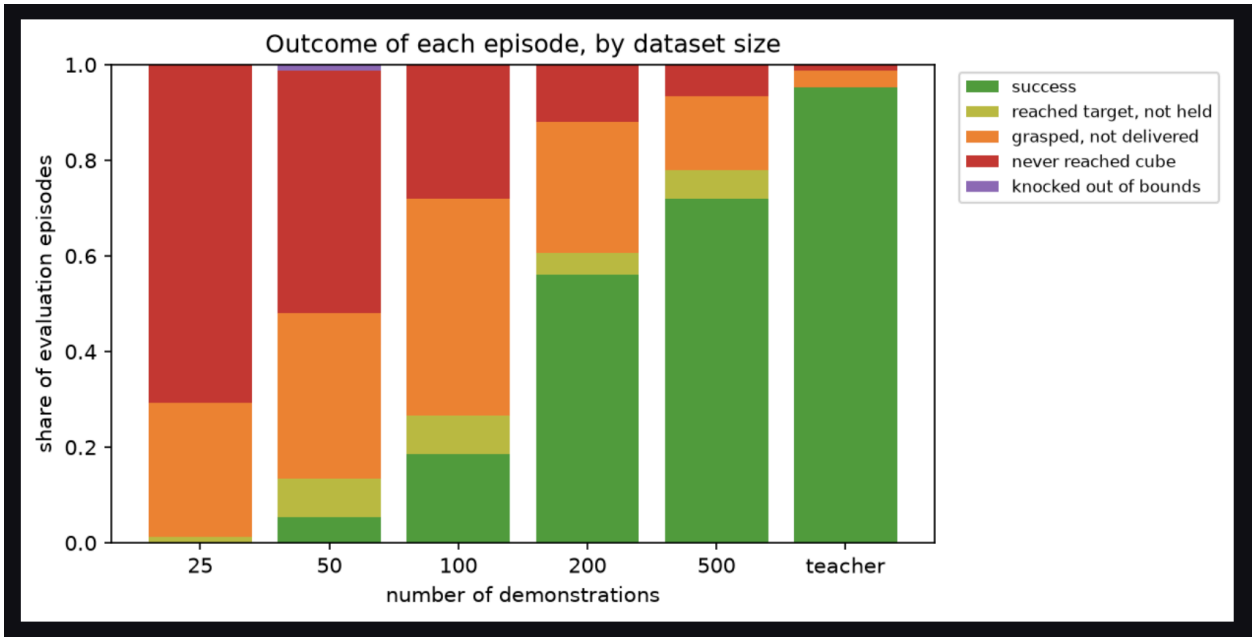


EVALUATION RESULTS - 150 rollouts over 3 seeds

Demos	Success (2 cm)	Ever reached target	Reached cube	Median final dist
25	0.0%	1.3%	29.3%	33.9 cm
50	5.3%	13.3%	49.3%	17.6 cm
100 (baseline)	**18.7%**	**26.7%**	**72.0%**	**4.6 cm**
200	56.0%	60.7%	88.0%	1.7 cm
500	72.0%	78.0%	93.3%	1.1 cm
Teacher	95.3%	95.3%	98.7%	0.48 cm

As the demonstrations increase you can clearly see an increase in success rates, jumping the most from 100 to 200 demos when the median final distance drops below the 0.02m threshold. Success in reaching the box improves steadily with more demos (29 → 49 → 72 → 88 → 93%).


Success vs dataset size (150 episodes per point, 95% CI)
fraction of evaluation episodes
1.0
0.8
0.6
0.4
0.2
0.0
25
50
100
200
500
number of demonstrations (log scale)
teacher (0.95, n=150)
student success


Outcome of each episode, by dataset size
share of evaluation episodes
1.0
0.8
0.6
0.4
0.2
0.0
25
50
100
200
500
teacher
number of demonstrations
success
reached target, not held
grasped, not delivered
never reached cube
knocked out of bounds

FAILURE ANALYSIS -
With few demos, failures happen early: “never reached the cube” accounts for 71% of episodes at 25 demos, falling to 28% at 100 and 7% at 500. “Reached but not delivered” peaks at 100 demos (45%) and then falls to 15% at 500, where it is the largest remaining failure. “Reached the target but not held” stays at 5–8% from 50 to 500 demos and is 0% for the teacher, so more demos did not fix the drift.
When the robot reaches the cube but fails to grasp it, the robot enters states the teacher never produced. In these states the policy’s outputs aren’t grounded in any training data, and it doesn’t know how to recover. This is the compounding-error problem of behavior cloning discussed in class where small mistakes lead to unfamiliar states, where mistakes grow. More demos reduce it by covering more of the states the policy can reach.

REPRODUCIBILITY AND REFLECTION -
Easiest way to reproduce the results is to
git clone https://github.com/vaidehidalmia/pick_cube_teacher_student_policy.git
(Teacher checkpoint and the demos are included in the repo)
uv sync
source .venv/bin/activate
for s in 1000 2000 3000; do python code/eval_teacher.py --eval-seed $s; done
bash code/run_all.sh - will train and evaluate the student policies on 3 different seeds
python code/visualize_demos.py
python code/plot_vs_size.py

Everything apart from the student policies are saved in the git repo. The student policies were too big to save.

With more engineering time I would try DAgger. I am also curious to see after how much data does the policy stop improving.

CITATIONS -
I used Claude to help understand the existing code that I found. I asked it to check my code, writeup. It helped me write the bash script and visualization scripts.

What does Jax.random.split give

Explaining how JAX handles randomness with stateless keys.

`jax.random.split(key)` takes one random key and returns **two new, independent keys.** You can also ask for more: `jax.random.split(key, 5)` returns 5.

Why JAX needs it

In NumPy, the random generator has hidden internal state. Each `np.random.rand()` call quietly advances it, so you get different numbers each time. JAX has no hidden state: **the same key always gives the same random numbers.** Calling `jax.random.uniform(key)` twice with the same key gives the identical value twice.

So to get *new* randomness, you make new keys. `split` is how you do that without losing reproducibility: everything still traces back to your one starting seed.

The usual pattern

python

```
rng = jax.random.PRNGKey(1000)          # one seed for the whole run
rng, reset_rng = jax.random.split(rng)  # keep one key, use the other
state = jit_reset(reset_rng)            # this episode's random start
```

okay lets start collecting demos, dont write the script, help me through it

Planning how to guide a step-by-step process without code.

Let's build `collect_demos.py` in four steps. Write each part and show it to me before moving on.

Step 1: Add a `collect` section to `config.yaml`

You'll need: the collection seed (0, different from teacher evaluation's 1000), the number of episodes (100), and where to save the dataset (somewhere under `data/`). Add `DATA = ROOT / "data"` to `common.py` the same way as `RESULTS`.

Step 2: The collection loop

It's your `evaluate` loop plus recording. Per episode, keep lists of:

- `obs`: `state.obs` **before** calling `jit_step`. This is the most important detail: each stored action must be paired with the observation it was chosen from.
- `actions`: the `ctrl` the teacher returned.
- `rewards` and `dists`: after the step, for diagnostics.

After each episode, compute the per-episode labels: `success`, `final_dist`, and `time_to_success` (the first step after which the cube stays within the threshold; -1 if it failed).

Two things to think about:

- **Deterministic teacher.** The only variety be[illegible]en demos comes from the random starting positions. That's fine, and worth a [illegible]ence in your consistency discussion.

lets walk through how that code works first

Explaining dataset preparation code for sequence sampling.

Here's how the Colab's dataset code works. It has five small pieces, and each one maps directly to your data.

The big picture

Each training sample is a **window of 16 consecutive timesteps** cut out of one episode. From that window, the model gets:

- **Observations:** only the first 2 steps (the observation horizon).
- **Actions:** all 16 steps (the prediction horizon). This is what the model learns to predict.

```
window index:  0    1    2    3   ...   15
obs used:      ✓    ✓
actions used:  ✓    ✓    ✓    ✓   ...   ✓
               t-1  t   t+1  t+2  ...  t+14
```

Index 1 is "now" (step *t*), and index 0 is the previous step. The action sequence also **starts at t−1**, not at *t*. That matters at evaluation: of the 16 predicted actions, you execute indices **1 through 8** (starting at "now"), not 0 through 7.

Piece 1: `create_sample_indices` (where can windows start?)

It loops over episodes using `episode_ends` and ↓ every valid window start. It allows windows to hang off the edges of an episode by a little:

explain create_sample_indices

Walking through the function's logic step by step.

`create_sample_indices` decides **which 16-step windows exist**, and for each one, **where its real data comes from** and **where padding is needed**. It doesn't copy any data; it only computes four numbers per window. `sample_sequence` uses those numbers later to cut out the actual window.

Inputs

- `episode_ends`: where each episode ends in the flat arrays, e.g. `[150, 300, ..., 15000]`.
- `sequence_length`: the window length, 16 (the prediction horizon).
- `pad_before = 1` and `pad_after = 7`: how far a window may hang off the start and end of an episode.

Step 1: loop over episodes

```python
for i in range(len(episode_ends)):
    start_idx = 0 if i == 0 else episode_ends[i-1]
    end_idx = episode_ends[i]
    episode_length = end_idx - start_idx
```

For each episode, work out where it sits in the flat arrays. Episode 0 is rows 0–149, episode 1 is rows 150–299, and so on.

Step 2: the range of window starts

give me an example for select_episodes so i can think about how to code it

Here's a toy example small enough to work through by hand.

The full dataset: 3 episodes of different lengths

```
row:          0   1   2   3 | 4   5   6 | 7   8   9   10  11
episode:      0   0   0   0 | 1   1   1 | 2   2   2   2   2
obs (1-dim): a0  a1  a2  a3 | b0  b1  b2| c0  c1  c2  c3  c4

episode_ends = [4, 7, 12]
```

(Your real episodes are all 150 steps, but different lengths make the example clearer, and your code should handle it anyway: a student policy's episodes could end early.)

Select episodes `[0, 2]`

Questions to answer for each selected episode:

1. **Where does it start?** Episode 0 starts at 0. Episode 2 starts at `episode_ends[1]` = 7.
2. **Where does it end?** Episode 0 ends at `episode_ends[0]` = 4. Episode 2 ends at `episode_ends[2]` = 12.
3. **Which rows?** Episode 0: rows 0–3 (`obs[0:4]`). Episode 2: rows 7–11 (`obs[7:12]`).
4. **How long?** 4 and 5.

The result

```
new row:   0   1   2   3 | 4   5   6   7   8
obs:      a0  a1  a2  a3 | c0  c1  c2  c3  c4
```